

【专栏二：深度学习08】 - 【一张图讲清楚：为什么 ReLU 也不是完美的？什么是死亡 ReLU？】



快乐飞猪ing



· 码龄7年 · 粉丝可见 · 编辑



墨衍会员 · AI 创作全网分发

原创 · 2026-03-29 11:21:03 · 394 阅读



AI 写同款 · 一键分发



实战指南：利用Python可视化常见激活函数（Sigmoid、Tanh、ReLU、PReLU）及其特性...

[阅读详情 >](#)

本文提供了一份实战指南，详细讲解如何使用Python可视化Sigmoid、Tanh、ReLU和PReLU等常见激活...

前言

在[深度学习](#)中，ReLU 是最常用的激活函数之一。

我们都知道它的优点：

- 计算简单
- 收敛快
- 梯度不容易消失

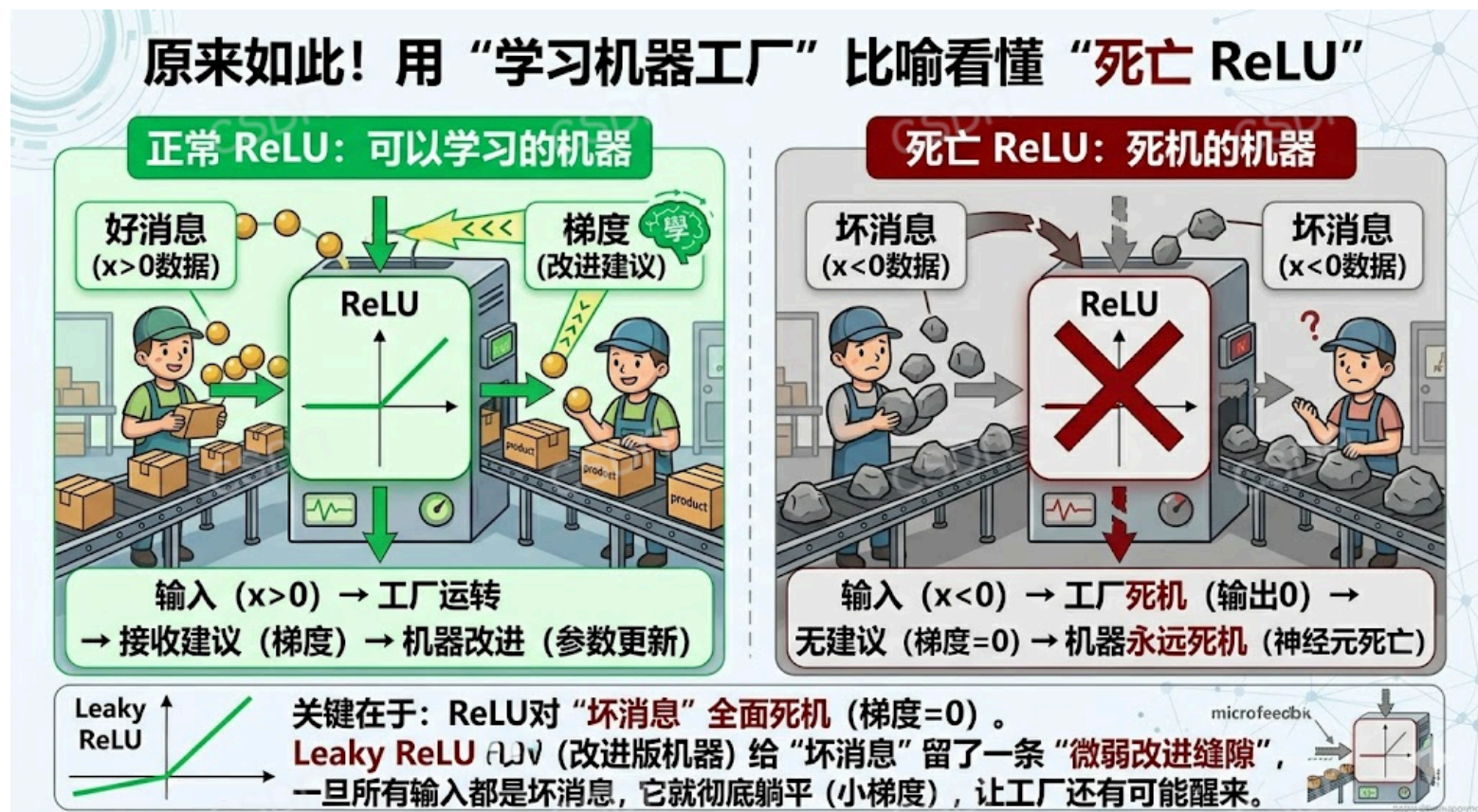
但问题来了：如果 ReLU 这么好，为什么模型有时候会“学着学着就不动了”？

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159588764

作者主页：https://blog.csdn.net/m0_46279060



这张图其实已经讲清楚了一个非常重要的对比：

左边：正常 ReLU → 可以持续学习

右边：死亡 ReLU → 完全停止学习

一、ReLU 到底在干嘛？

ReLU 的定义非常简单： $f(x) = \max(0, x)$

也就是说：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159588764

作者主页：https://blog.csdn.net/m0_46279060

- 当 $x > 0 \rightarrow$ 输出 x
- 当 $x \leq 0 \rightarrow$ 输出 0

这意味着：ReLU 会“屏蔽掉”所有负数输入

二、正常情况：神经元是“活着的”

看图左边。

当输入是正数 ($x > 0$) 时：

- 输出正常
- 梯度存在
- 可以反向传播

训练过程是这样的：输入 $\rightarrow$ ReLU $\rightarrow$ 输出 $\rightarrow$ 计算损失 $\rightarrow$ 反向传播 $\rightarrow$ 更新参数

关键点：

有梯度 $\rightarrow$ 参数可以更新 $\rightarrow$ 模型在学习

三、问题出现：什么时候会“死掉”？

看图右边。

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159588764

作者主页：https://blog.csdn.net/m0_46279060

当输入长期是负数 ($x < 0$) 时:

- 输出恒为 0
- 梯度 = 0

用公式说就是:

$$x < 0 \Rightarrow f(x) = 0, \frac{\partial f}{\partial x} = 0$$

这会带来一个非常严重的问题: **没有梯度** → **无法反向传播** → **参数无法更新**

👉 结果就是: **这个神经元彻底失去学习能力**

四、为什么一旦“死了”就回不来?

这是很多人没意识到的关键点。

假设某个神经元进入了负区间:

```
go AI写代码 | ▶ 运行  
1 Iteration 1:  
2  $x < 0 \rightarrow$  输出 0  $\rightarrow$  梯度 = 0  
3  
4 Iteration 2:  
5 参数不变  $\rightarrow$  仍然  $x < 0$   
6  
7  
8
```

内容来源: csdn.net
作者昵称: 快乐飞猪ing
原文链接: https://blog.csdn.net/m0_46279060/article/details/159588764
作者主页: https://blog.csdn.net/m0_46279060

👉 形成一个死循环：没有梯度 → 参数不变 → 还是负数 → 永远没有梯度

这就是：死亡 ReLU

五、为什么会发生死亡 ReLU?

常见有三个原因：

1. 学习率过大

一次更新把参数“打到负区间”，再也回不来。

2. 初始化不合理

一开始权重就偏负，神经元直接“出生即死亡”。

3. 数据分布问题

输入本身偏负，ReLU 一直不激活。

六、影响是什么?

死亡 ReLU 并不会让训练直接报错，但会带来隐性问题：

- 一部分神经元完全失效
- 网络有效容量下降

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159588764

作者主页：https://blog.csdn.net/m0_46279060

- 表达能力变弱
- 训练变慢甚至停滞

一句话总结：

模型还在训练，但有些神经元已经“挂机了”。

七、怎么解决？

最常见的方法是：

Leaky ReLU

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

AI写代码 |

1 |

区别在于：

- ReLU：负区间直接归零（梯度=0）
- Leaky ReLU：负区间保留一点斜率（梯度≠0）

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159588764

作者主页：https://blog.csdn.net/m0_46279060

👉 这意味着：

即使输入是负的，神经元也还有“微弱的学习能力”

我们可以把这件事总结成一句话：**ReLU 的强大来自于“非线性”，但它的风险也来自于“零梯度”。**

再压缩一句：

没有梯度，就没有学习。

ReLU 会“死”，但它依然是最常用的[激活函数](#)。

是因为：

- 它的问题是“可控的”
- 它的优点远大于缺点

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159588764

作者主页：https://blog.csdn.net/m0_46279060